Indian Institute of Technology Dharwad

CS201L: Artificial Intelligence Laboratory

Lab 1: Introduction to Python Programming and Data Science

Date: 08-01-2026

Problem Statement 1

A residential housing dataset is provided in a CSV file (`residential_housing.csv`). The dataset contains various attributes that describe the physical, locational, and quality-related characteristics of houses. These features are:

Table 1: Description of Attributes in the Residential Housing Dataset

Feature	**Description**
Id	Unique identifier assigned to each property record.
MSSubClass	Type of dwelling involved in the transaction.
MSZoning	General zoning classification of the property.
LotFrontage	Linear feet of street connected to the property.
LotArea	Total lot size in square feet.
Street	Type of road access to the property.
Alley	Type of alley access to the property.
LotShape	General shape of the property lot.
LandContour	Flatness of the property.
Utilities	Type of utilities available at the property.
LotConfig	Lot configuration.
LandSlope	Slope of the property.
Neighborhood	Physical location within the city.
Condition1	Proximity to main road or railroad.
Condition2	Secondary proximity to main road or railroad.
BldgType	Type of dwelling.
HouseStyle	Style of dwelling.
OverallQual	Overall material and finish quality.
OverallCond	Overall condition rating.
YearBuilt	Original construction year.
YearRemodAdd	Year of remodeling or addition.
RoofStyle	Type of roof.
RoofMatl	Roof material.
Exterior1st	Primary exterior covering.

Feature	Description
Exterior2nd	Secondary exterior covering.
MasVnrType	Masonry veneer type.
MasVnrArea	Masonry veneer area in square feet.
ExterQual	Exterior material quality.
ExterCond	Exterior material condition.
Foundation	Type of foundation.
BsmtQual	Basement height quality.
BsmtCond	Basement condition.
BsmtExposure	Basement exposure level.
BsmtFinType1	Primary basement finished area type.
BsmtFinSF1	Size of primary finished basement area.
BsmtFinType2	Secondary basement finished area type.
BsmtFinSF2	Size of secondary finished basement area.
BsmtUnfSF	Unfinished basement area.
TotalBsmtSF	Total basement area.
Heating	Type of heating.
HeatingQC	Heating quality and condition.
CentralAir	Central air conditioning availability.
Electrical	Electrical system type.
1stFlrSF	Area of first floor in square feet.
2ndFlrSF	Area of second floor in square feet.
LowQualFinSF	Low quality finished living area.
GrLivArea	Above ground living area.
BsmtFullBath	Number of basement full bathrooms.
BsmtHalfBath	Number of basement half bathrooms.
FullBath	Number of full bathrooms above ground.
HalfBath	Number of half bathrooms above ground.
BedroomAbvGr	Number of bedrooms above ground.
KitchenAbvGr	Number of kitchens above ground.
KitchenQual	Kitchen quality rating.
TotRmsAbvGrd	Total number of rooms above ground.
Functional	Home functionality rating.
Fireplaces	Number of fireplaces.
FireplaceQu	Fireplace quality.
GarageType	Garage location type.
GarageYrBlt	Year garage was built.
GarageFinish	Garage interior finish.
GarageCars	Number of vehicles the garage can hold.
GarageArea	Size of garage in square feet.

Feature	Description
GarageQual	Garage quality.
GarageCond	Garage condition.
PavedDrive	Type of paved driveway.
WoodDeckSF	Wood deck area in square feet.
OpenPorchSF	Open porch area.
EnclosedPorch	Enclosed porch area.
3SsnPorch	Three-season porch area.
ScreenPorch	Screen porch area.
PoolArea	Pool area in square feet.
PoolQC	Pool quality rating.
Fence	Fence quality.
MiscFeature	Miscellaneous feature not covered in other categories.
MiscVal	Value of miscellaneous feature.
MoSold	Month of sale.
YrSold	Year of sale.
SaleType	Type of sale.
SaleCondition	Condition of sale.
SalePrice	Recorded sale price of the property.

Write a Python program to perform the following tasks:

1. Write a `for` loop to count how many houses in the dataset have a `LotArea` greater than 2000 sq.ft.
2. Write a `for` loop to count how many houses have `BedroomAbvGr` equal to 3 in the dataset.
3. Write a `while` loop to print the `LotArea` of the first 10 houses in the dataset.
4. Using a `while` loop, find the total `SalePrice` of the first 15 houses in the dataset.
5. Create a `list` containing the `LotArea` of the first 10 houses and display the maximum value from the list.
6. Create a `list` of `SalePrice` values of houses where `BedroomAbvGr` is equal to 3.
7. Create a `tuple` containing the first 5 houses' `(LotArea, SalePrice)` pairs.
8. Create a `tuple` of `BedroomAbvGr` values for the first 10 houses and find how many times the value 2 occurs.
9. Create a `dictionary` where `LotArea` is the key and `SalePrice` is the value for the first 10 houses.
10. Create a `dictionary` to count how many houses belong to each Neighborhood category.

11. Using `numpy.mean()`, compute the mean value of the `SalePrice` attribute.
12. Determine the minimum and maximum values of the `LotArea` attribute using `numpy.min()` and `numpy.max()`.
13. Find the index of the house with the highest `SalePrice` using `numpy.argmax()`, and display the complete record corresponding to that index.
14. Using `numpy.where()`, identify all houses whose `GrLivArea` is greater than 2000 square feet.
15. Sort the `SalePrice` attribute in descending order using `numpy.sort()` and display the top ten values.
16. Determine the number of unique `Neighborhood` categories present in the dataset using `numpy.unique()`.
17. Generate five equally spaced bins between the minimum and maximum `YearBuilt` values using `numpy.linspace()`.
18. Using NumPy vectorized operations (such as boolean masking and `numpy.sum()`), calculate the total number of houses built after the year 2000.
19. Compute the standard deviation of the `GarageArea` attribute using `numpy.std()`.
20. Create a new NumPy array that labels houses as "Large" if `LotArea` > 10,000 and "Small" otherwise using `numpy.where()`.
21. Convert the given CSV file to a DataFrame and display the first and last 10 tuples using the `head()` and `tail()` functions.
22. Determine the total number of records and attributes present in the dataset.
23. Display the structure of the dataset using the `info()` function.
24. Generate descriptive statistics using the `describe()` function.
25. Compute the average `SalePrice` for each `Neighborhood` using grouping operations.
26. Display the five houses with the highest `SalePrice`.
27. Count the number of houses built after the year 2000.

Problem Statement 2

A real-world medical dataset containing hospital appointment records is provided in a CSV file (`noshowappointments.csv`). The dataset includes various attributes that provide insights into patient appointment behavior. These features are:

Table 2: Description of Attributes in the Patient Appointment Dataset

Feature	**Description**
PatientId	Unique identifier assigned to each patient.
AppointmentID	Unique identifier assigned to each appointment record.
Gender	Gender of the patient.
ScheduledDay	Date on which the appointment was scheduled.
AppointmentDay	Actual date of the appointment.
Age	Age of the patient in years.
Neighbourhood	Residential area where the patient lives.
Scholarship	Indicates whether the patient is enrolled in a social welfare program.
Hypertension	Indicates whether the patient has hypertension.
Diabetes	Indicates whether the patient has diabetes.
Alcoholism	Indicates whether the patient has a history of alcoholism.
SMS_received	Indicates whether the patient received a reminder SMS.
No-show	Indicates whether the patient failed to attend the appointment.

Write a Python program to perform the following tasks:

1. Write a `for` loop to count how many patients have `SMS_received` equal to 1.
2. Write a `for` loop to count how many patients have `Hypertension` equal to 1.
3. Write a `while` loop to print the `Age` of the first 10 patients.
4. Using a `while` loop, find the total `Age` of the first 15 patients in the dataset.
5. Create a `list` containing the `Age` of the first 10 patients and display the minimum age from the list.
6. Create a `tuple` containing the first 5 patients' `(Age, SMS_received)` pairs.
7. Create a `dictionary` to count how many patients belong to each Gender category.
8. Using `numpy.median()`, compute the median value of the `Age` attribute for patients in the dataset.
9. Count the number of patients who are older than 60 years using vectorized Boolean operations.
10. Determine the proportion of patients who received an SMS reminder (`SMS_received = 1`).
11. Extract the first 50 appointment records using NumPy indexing.
12. Compute the variance of the `Age` attribute using NumPy.

13. Use `numpy.unique()` to count the number of unique neighborhoods in the dataset.
14. Generate a NumPy array of ten evenly spaced threshold values between the minimum and maximum `Age` values using `numpy.linspace()`.
15. Create a NumPy array that encodes `No-show` as 1 if the patient missed the appointment and 0 otherwise using vectorized conditional operations.
16. Display the first and last 10 tuples of the given dataset using the `head()` and `tail()` functions.
17. Display the structure of the data using the `info()` function.
18. Determine the total number of appointments scheduled in each `Neighbourhood`.
19. Calculate the percentage of appointments that resulted in a no-show.

Note

Kindly upload the completed Jupyter Notebook for each of the problem statements separately to Moodle for evaluation.

Deliverables

- Python scripts implementing all specified tasks.
- Visualizations generated for scatter plots, histograms, bar plots, and boxplots.
- A summary of findings from the statistical analysis and visualizations.

References

[1] P. Cortez, A. Cerdeira, F. Almeida, T. Matos, and J. Reis, "Modeling wine preferences by data mining from physicochemical properties," *Decision Support Systems*, vol. 47, no. 4, pp. 547–553, 2009.